
Writing CUDA Kernels From Scratch

A construction manual for the physics programmer

From first principles to production-quality batched reduction kernels

This manual is written to be read *linearly* on first pass. Each section builds directly on the previous one. By the end you should be able to look at a kernel like `cov_reduce_sym_warp` and not only understand every line, but understand the sequence of decisions that led to writing it that way rather than any other way. The goal is not to memorise patterns — it is to internalise the reasoning so you can derive the patterns yourself.

Contents

1	The Mental Model: What a GPU Actually Is	2
1.1	A GPU is a latency-tolerant throughput machine	2
1.2	The three-level thread hierarchy	2
1.3	The memory hierarchy	3
1.4	How threads know who they are	3
2	Your First Kernel: Parallel Vector Addition	3
2.1	The kernel	3
2.2	Anatomy of a kernel signature	4
2.3	Launching from Python with CuPy	4
3	Pointers and Memory in C: What You Must Know	5
3.1	A pointer is just an address	5
3.2	Arrays are contiguous memory; array indexing is pointer arithmetic	5
3.3	Pointer arithmetic scales by type size	6
3.4	Row-major 2D array indexing	6
3.5	Navigating a batched output array	6
4	The Parallel Reduction Problem	7
4.1	The naive approach and why it fails	7
4.2	The strided accumulation pattern	7
4.3	Tree reduction in shared memory	7
5	Warp Shuffle Reduction: The Modern Approach	8
5.1	What shuffle instructions are	8
5.2	Building warp_sum step by step	8
5.3	Extending to a full block: the two-level reduction	9

6	Batched Reduction: The edges Pattern	10
6.1	The problem: many independent reductions over irregular segments	10
6.2	The CSR edges pattern	10
6.3	Memory coalescing in the strided loop	11
7	Accumulating Multiple Outputs Simultaneously	11
7.1	The problem: coupled outputs sharing expensive computation	11
7.2	The multi-accumulator pattern	12
7.3	Exploiting symmetry to reduce the problem size	12
8	Runtime Kernel Templating with Python f-Strings	13
8.1	The problem: compile-time constants for loop unrolling	13
8.2	The <code>make_kernel_source</code> pattern	13
9	Putting It All Together: Building the Full Kernel	14
9.1	Step 1: Define the computation	14
9.2	Step 2: Identify the parallelism	15
9.3	Step 3: Design the data layout	15
9.4	Step 4: Plan the reduction pipeline	15
9.5	Step 5: Size the shared memory	15
9.6	Step 6: Write and annotate the kernel	15
9.7	Step 7: Write the Python wrapper	17
10	Production Deployment: <code>nvcc</code>, Shared Libraries, and <code>ctypes</code>	18
10.1	The file structure	18
10.2	The CUDA file: kernel plus CPU launcher	19
10.3	The header file	21
10.4	Compiling with <code>nvcc</code>	22
10.5	Loading and calling from Python via <code>ctypes</code>	23
10.6	What <code>.data.ptr</code> is	24
10.7	Comparison: CuPy RawKernel vs <code>nvcc</code> + <code>ctypes</code>	25
11	Production Templating: <code>nvcc -D</code> and C Preprocessor Templates	25
11.1	Approach 1: <code>-D</code> flags at compile time	25
11.2	Approach 2: C++ templates for compile-time generics	26
11.3	Comparing the two templating approaches	29
11.4	The <code>constexpr</code> keyword	29
12	Performance Checklist and Common Mistakes	29
12.1	Checklist before calling a kernel production-ready	29
12.2	Common mistakes	30
13	Quick Reference: Kernel Design Recipes	31

1. The Mental Model: What a GPU Actually Is

Before writing a single line of code, you need a hardware mental model that is accurate enough to reason about performance. This section builds that model.

1.1. A GPU is a latency-tolerant throughput machine

A CPU is designed to minimise **latency** — the time it takes to complete one task. It achieves this with deep pipelines, branch predictors, out-of-order execution, and large caches. A single CPU core is extraordinarily good at running one complex task as fast as possible.

A GPU is designed to maximise **throughput** — the number of tasks completed per second across the entire chip. It achieves this by running thousands of simple tasks simultaneously, and by hiding the latency of any one task by switching to another whenever the first stalls. Individual GPU threads are slow and simple; the aggregate is enormously powerful.

This difference has a concrete consequence for how you write code. On a CPU you optimise a single thread. On a GPU you design for massive parallelism from the outset — the question is always “how do I express this computation so that thousands of threads can work on it simultaneously?”

1.2. The three-level thread hierarchy

Every CUDA program is built on three nested levels of organisation. You must always know which level you are thinking about.

Thread

The thread is the smallest unit. It executes a stream of instructions, has its own private **registers** (the fastest storage on the GPU, ~ 1 cycle latency), and has a unique identity given by `threadIdx.x` (and `.y`, `.z` for 2D/3D problems). Every variable you declare inside a kernel — `float acc`, `int lane`, etc. — lives in a thread’s private registers. No other thread can see them.

Warp

The warp is the fundamental unit of **hardware execution**. A warp is always exactly 32 threads. The hardware schedules and executes all 32 threads of a warp together, running the same instruction at the same clock cycle on different data. This is called **SIMT** (Single Instruction, Multiple Threads).

This has two important consequences. First, if threads within a warp take different branches (e.g. half satisfy an `if` condition and half do not), the warp must execute *both* branches sequentially, masking off the inactive threads — this is **warp divergence** and wastes execution capacity. Second, threads within a warp can exchange register values directly using **shuffle instructions**, without touching memory at all.

Block

A block is a group of warps (and thus threads) that share a region of fast on-chip memory called **shared memory** (~ 10 cycle latency). All threads within a block can read and write shared memory. Threads in different blocks cannot communicate directly. You choose the block size when launching a kernel; typical values are 128 or 256 threads.

Grid

The grid is all the blocks launched by one kernel call. You also choose the grid size at launch. For a batched problem with N independent groups, you typically launch N blocks — one per group.

1.3. The memory hierarchy

This is the most important table in GPU programming. Burn it into your memory.

Memory	Latency	Scope	Programmer control
Registers	~1 cycle	Per thread	Implicit (local variables)
Shared memory	~10 cycles	Per block	Explicit (<code>__shared__</code>)
L1/L2 cache	~30–100 cycles	Device-wide	Automatic
Global (VRAM)	~400–800 cycles	Device-wide	Explicit (kernel arguments)

The central challenge of GPU programming is: keep your hot data in registers and shared memory, and minimise round-trips to global VRAM. Every performance optimisation in a well-written kernel is ultimately in service of this goal.

1.4. How threads know who they are

Every thread has hardware-readable identity registers that are always available:

```
threadIdx.x    // this thread's index within its block (0 to blockDim.x-1)
blockIdx.x     // this block's index within the grid (0 to gridDim.x-1)
blockDim.x     // total number of threads per block
gridDim.x      // total number of blocks in the grid
```

Listing 1. Built-in identity variables available in every kernel

These are not function arguments. They are hardware registers, always present, always correct, available everywhere including inside `__device__` helper functions. A thread's identity never changes for the duration of the kernel.

The standard idiom for computing a unique global thread index is:

```
int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
```

Listing 2. Standard idiom: unique global thread index

2. Your First Kernel: Parallel Vector Addition

The simplest meaningful kernel adds two arrays element-wise. It introduces the complete anatomy of a CUDA kernel.

2.1. The kernel

```
1  /* The __global__ qualifier marks this as a kernel ---
2     called from CPU, executed on GPU. */
3  extern "C" __global__
```

```

4 void vec_add(const float* __restrict__ A,
5             const float* __restrict__ B,
6             float*      __restrict__ C,
7             int N)
8 {
9     /* Each thread computes one output element. */
10    int i = blockIdx.x * blockDim.x + threadIdx.x;
11
12    /* Bounds check: the grid may be larger than N. */
13    if (i >= N) return;
14
15    C[i] = A[i] + B[i];
16 }

```

Listing 3. Hello World of GPU programming: parallel vector addition

2.2. Anatomy of a kernel signature

- `extern "C"` — tells the compiler to use C-style name mangling. Required when calling kernels from Python via CuPy.
- `__global__` — this function is a kernel. It is called from the CPU (host) and runs on the GPU (device).
- `const float* __restrict__ A` — a pointer to read-only float data. `const` tells the compiler you will not modify what `A` points to. `__restrict__` tells the compiler that `A`, `B`, and `C` do not overlap in memory, enabling more aggressive optimisation.
- `int N` — a scalar value passed by copy.

2.3. Launching from Python with CuPy

```

1 import cupy as cp
2 import numpy as np
3
4 # Define the kernel source
5 kernel_source = r'''
6 extern "C" __global__
7 void vec_add(const float* __restrict__ A,
8             const float* __restrict__ B,
9             float*      __restrict__ C,
10             int N)
11 {
12     int i = blockIdx.x * blockDim.x + threadIdx.x;
13     if (i >= N) return;
14     C[i] = A[i] + B[i];
15 }
16 '''
17
18 # Compile once (cached after first call)
19 kernel = cp.RawKernel(kernel_source, "vec_add")
20
21 # Allocate data on GPU
22 N = 1024
23 A = cp.ones(N, dtype=cp.float32)
24 B = cp.ones(N, dtype=cp.float32) * 2.0
25 C = cp.empty(N, dtype=cp.float32)
26

```

```

27 # Launch configuration
28 threads_per_block = 128
29 blocks = (N + threads_per_block - 1) // threads_per_block # ceiling
      division
30
31 # Launch: (grid,), (block,), (args...)
32 kernel((blocks,), (threads_per_block,), (A, B, C, np.int32(N)))
33
34 # C is now [3.0, 3.0, ..., 3.0]

```

Listing 4. Launching the kernel from Python

`(N + threads_per_block - 1) // threads_per_block` is the standard way to compute the number of blocks needed to cover `N` elements when `N` may not be divisible by `threads_per_block`. This is why kernels always start with a bounds check `if (i >= N) return` — some threads in the last block may have indices beyond the array.

3. Pointers and Memory in C: What You Must Know

CUDA C is C. A solid understanding of C pointers is non-negotiable for writing kernels. This section covers exactly what you need.

3.1. A pointer is just an address

A pointer is a variable that holds a memory address — a number indicating where in memory some data lives. That is all it is. There is no magic.

```

1 float x = 3.14f;
2 float* p = &x;           // p holds the address of x
3 float y = *p;            // dereference: read the float at address p
4 *p = 2.71f;              // write the float at address p (modifies x)

```

Listing 5. Pointers are addresses

3.2. Arrays are contiguous memory; array indexing is pointer arithmetic

In C, an array is simply a contiguous block of memory. The array name is a pointer to the first element. Indexing is defined as pointer arithmetic:

```

1 float arr[6] = {1, 2, 3, 4, 5, 6};
2
3 /* These are all identical: */
4 float val = arr[2];      // standard indexing
5 float val = *(arr + 2);  // explicit pointer arithmetic
6 /* arr + 2 means: advance 2 * sizeof(float) = 8 bytes from arr's address */

```

Listing 6. Array indexing is pointer arithmetic

When you write `C[i] = A[i] + B[i]` in a kernel, the compiler translates this to pointer arithmetic and memory loads/stores. There is no bounds checking, no reference counting, no garbage collection. If `i` is out of bounds, the kernel silently reads or writes memory that

does not belong to your array.

3.3. Pointer arithmetic scales by type size

Adding an integer n to a pointer of type `T*` advances it by $n \times \text{sizeof}(T)$ bytes — not by n bytes. The compiler handles the scaling automatically.

```

1 float* p = some_array;
2 p + 1;    // advance by 4 bytes (sizeof(float) = 4)
3 p + 10;   // advance by 40 bytes
4
5 int* q = some_int_array;
6 q + 1;    // advance by 4 bytes (sizeof(int) = 4)
7
8 double* d = some_double_array;
9 d + 1;    // advance by 8 bytes (sizeof(double) = 8)

```

Listing 7. Pointer arithmetic scales by element size

3.4. Row-major 2D array indexing

A 2D array of shape (M, N) stored in **row-major** (C) order lays out rows contiguously: row 0 first, then row 1, etc. The element at row r , column c is at flat index $r \times N + c$:

```

1 /* A (3,3) matrix stored as a flat 9-element array */
2 float mat[9]; /* mat[r*3 + c] = element at row r, col c */
3
4 mat[0*3 + 0] = 1.0f; /* (0,0) */
5 mat[0*3 + 1] = 2.0f; /* (0,1) */
6 mat[0*3 + 2] = 3.0f; /* (0,2) */
7 mat[1*3 + 0] = 4.0f; /* (1,0) */
8 /* ... etc */
9
10 /* For a general (n_eig x n_eig) matrix: */
11 mat[row * n_eig + col] /* element at (row, col) */

```

Listing 8. 2D row-major indexing — critical for understanding kernel output layout

This is the exact formula used in the output packing step of the covariance reduction kernel.

3.5. Navigating a batched output array

When you have a batch of B matrices each of size $M \times N$, stored contiguously, the element at matrix b , row r , column c is at flat index $b \times (M \times N) + r \times N + c$. In the kernel:

```

1 /* C_out is a flat array of nb * n_eig * n_eig floats.
2    To get a pointer to block b's n_eig x n_eig matrix: */
3 float* out = C_out + (size_t)b * N_EIG * N_EIG;
4
5 /* Now out[row * N_EIG + col] is element (row,col) of block b's matrix.
6    The (size_t) cast forces 64-bit arithmetic to prevent overflow
7    if b * N_EIG * N_EIG ever becomes large. */

```

Listing 9. Computing a pointer to block b 's output matrix

4. The Parallel Reduction Problem

The most fundamental operation in GPU computing beyond element-wise work is **reduction**: taking an array of N values and producing a single value (a sum, max, etc.). This section builds up the full reduction pattern from scratch.

4.1. The naive approach and why it fails

The CPU approach to summing an array is a serial loop:

```

1  /* BAD on GPU: one thread does everything, 127 threads idle */
2  __global__ void naive_sum(const float* A, float* result, int N) {
3      if (threadIdx.x == 0) {
4          float s = 0.0f;
5          for (int i = 0; i < N; i++) s += A[i];
6          *result = s;
7      }
8  }
```

Listing 10. CPU-style sum: only one thread does any work

This is catastrophic on a GPU. 127 of 128 threads sit idle. All the latency hiding and parallelism the hardware offers is completely wasted.

4.2. The strided accumulation pattern

The correct first step is to distribute the work across all threads using the **grid-stride loop**:

```

1  __global__ void partial_sum(const float* A, float* partials, int N) {
2      int tid = threadIdx.x;
3      int T    = blockDim.x;
4      int b    = blockIdx.x;
5
6      float acc = 0.0f;
7
8      /* Thread tid processes elements tid, tid+T, tid+2T, ... */
9      for (int i = b * T + tid; i < N; i += gridDim.x * T) {
10         acc += A[i];
11     }
12
13     /* acc now holds this thread's partial sum.
14        We still need to combine all threads' partial sums. */
15     partials[b * T + tid] = acc;
16 }
```

Listing 11. Grid-stride loop: every thread contributes

Now every thread does work. But we still have $128 \times \text{num_blocks}$ partial sums that need to be combined into one. That combination is the reduction step.

4.3. Tree reduction in shared memory

The classic approach uses shared memory and a binary tree:

```

1  __global__ void tree_reduce(float* partials, float* result, int N) {
2      extern __shared__ float sh[];
3      int tid = threadIdx.x;
4
```

```

5      /* Load one value per thread into shared memory */
6      sh[tid] = (tid < N) ? partials[tid] : 0.0f;
7      __syncthreads();
8
9      /* Binary tree reduction: at each step, half the threads add
10     the value from a thread stride away */
11     for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
12         if (tid < stride) {
13             sh[tid] += sh[tid + stride];
14         }
15         __syncthreads(); /* all threads must see the update */
16     }
17
18     /* Thread 0 holds the final sum */
19     if (tid == 0) *result = sh[0];
20 }

```

Listing 12. Tree reduction in shared memory

This works, but uses a lot of shared memory and requires $\log_2(N)$ synchronisation barriers. The modern approach using warp shuffles is both simpler and faster.

5. Warp Shuffle Reduction: The Modern Approach

5.1. What shuffle instructions are

Warp shuffle instructions allow threads within the same warp to directly read each other's registers without going through shared memory. Since all 32 threads in a warp execute in lock-step, this is safe without any synchronisation barrier.

The key instruction:

```

float result = __shfl_down_sync(mask, value, delta);
/* Each thread receives the value held by the thread delta positions
ahead of it in the warp. The mask specifies which lanes participate
(0xffffffff = all 32). */

```

Listing 13. The shuffle-down instruction

5.2. Building warp_sum step by step

To sum 32 values (one per lane), we apply shuffle-down with decreasing strides. At each step, every active thread adds the value from the thread `delta` positions ahead:

```

1  __device__ __forceinline__ float warp_sum(float v) {
2      unsigned mask = 0xffffffff; /* all 32 lanes */
3      v += __shfl_down_sync(mask, v, 16); /* step 1: pair threads 0-15 with
4      16-31 */
5      v += __shfl_down_sync(mask, v, 8); /* step 2: pair threads 0-7 with
6      8-15 */
7      v += __shfl_down_sync(mask, v, 4); /* step 3: pair threads 0-3 with
8      4-7 */
9      v += __shfl_down_sync(mask, v, 2); /* step 4: pair threads 0-1 with
10     2-3 */
11     v += __shfl_down_sync(mask, v, 1); /* step 5: pair thread 0 with
12     1 */
13     return v;

```

```

9      /* After step 5, lane 0 holds the sum of all 32 values.
10         No shared memory used. No __syncthreads() needed.
11         Total: 5 hardware instructions. */
12  }
```

Listing 14. warp_sum: reduce 32 values to 1 in 5 shuffle instructions

This function is called once per kernel and its body is tiny. The compiler would likely inline it anyway, but `__forceinline__` makes it a guarantee rather than a suggestion. After inlining, all 5 shuffle instructions appear directly in the calling kernel’s instruction stream where the compiler can schedule them optimally.

5.3. Extending to a full block: the two-level reduction

A warp has 32 threads, but a block may have 128 or 256. After each warp runs `warp_sum`, we have one partial sum per warp. A second reduction pass across warps is needed.

```

1  __global__ void block_sum(const float* A, float* result, int N) {
2      /* ---- Phase 1: each thread accumulates its own partial sum ---- */
3      float acc = 0.0f;
4      int tid = threadIdx.x;
5      int T    = blockDim.x;
6      for (int i = blockIdx.x * T + tid; i < N; i += blockDim.x * T)
7          acc += A[i];
8
9      /* ---- Phase 2: intra-warp reduction (fast, no shared memory) ---- */
10     acc = warp_sum(acc);
11     /* Now lane 0 of each warp holds that warp's partial sum. */
12
13     /* ---- Phase 3: write per-warp results to shared memory ---- */
14     int lane      = threadIdx.x & 31;      /* position within warp: x % 32 */
15     int warp      = threadIdx.x >> 5;      /* warp index: x / 32 */
16     int num_warps = blockDim.x >> 5;
17
18     extern __shared__ float sh[];          /* size: num_warps floats */
19     if (lane == 0) sh[warp] = acc;
20     __syncthreads();                      /* ensure all warps have written */
21
22     /* ---- Phase 4: warp 0 does the final reduction ---- */
23     if (warp == 0) {
24         /* Lane 0..num_warps-1 load the per-warp sums; rest load 0 */
25         float v = (lane < num_warps) ? sh[lane] : 0.0f;
26         v = warp_sum(v);
27         if (lane == 0) *result = v;        /* lane 0 writes final answer */
28     }
29 }
```

Listing 15. Two-level reduction: warp shuffle + shared memory handoff

This four-phase structure — **accumulate**, **warp-reduce**, **share**, **block-reduce** — is the backbone of every serious reduction kernel. You will see it again in the covariance kernel.

6. Batched Reduction: The edges Pattern

6.1. The problem: many independent reductions over irregular segments

In many physics problems you do not want one reduction over a single array. You want *many* reductions, each over a different segment of an array, where the segments may have different lengths. This is the case in radio interferometry: one reduction per baseline group, each group having a different number of time-frequency samples.

The naive solution — a Python loop launching one kernel per group — has enormous overhead. Each kernel launch takes ~ 10 – 50 microseconds. With 130,000 baseline groups, that alone costs several seconds.

6.2. The CSR edges pattern

The solution is to encode all segment boundaries in a single integer array and launch one block per segment:

```

1  /* edges has nb+1 entries.
2  Segment b spans rows edges[b] to edges[b+1]-1 (half-open interval).
3  This is identical to the 'indptr' array in scipy's csr_matrix. */
4
5  /* Example: 4 segments of sizes 47, 44, 49, 60 */
6  int edges[] = {0, 47, 91, 140, 200};
7  /*
8        |   |   |   |   total rows
9        |   |   |   |   segment 3 starts at 140
10       |   |   segment 2 starts at 91
11       |   segment 1 starts at 47
12       segment 0 starts at 0                */
```

Listing 16. CSR edges array: encoding irregular segment boundaries

```

1  extern "C" __global__
2  void batched_sum(const float* A,
3                  const int* edges,
4                  float* results,
5                  int nb)
6  {
7      int b = (int)blockIdx.x;      /* which segment this block owns */
8      if (b >= nb) return;
9
10     int start = edges[b];          /* first row of this segment */
11     int stop = edges[b + 1];       /* one past the last row */
12
13     /* Each thread in the block accumulates over its assigned rows */
14     float acc = 0.0f;
15     int tid = (int)threadIdx.x;
16     int T = (int)blockDim.x;
17
18     for (int i = start + tid; i < stop; i += T)
19         acc += A[i];
20
21     /* Two-level reduction (as above) ... */
22     acc = warp_sum(acc);
23     /* ... shared memory handoff ... */
24     /* ... final result written to results[b] */
25 }
```

Listing 17. Dispatching one block per segment

Each block reads only `edges[b]` and `edges[b+1]`, writes only `results[b]`, and accesses only rows `start` to `stop-1` of `A`. These ranges are guaranteed non-overlapping by construction of `edges`. There is no communication between blocks and no synchronisation needed across the grid.

6.3. Memory coalescing in the strided loop

The loop `for (int i = start + tid; i < stop; i += T)` is not just about distributing work — it is also about memory access efficiency.

On a warp’s first iteration, threads 0–31 access rows `start+0`, `start+1`, ..., `start+31`: 32 consecutive memory addresses, satisfied in a single 128-byte memory transaction. On the second iteration they access `start+T`, `start+T+1`, ...: again consecutive. This is **coalesced access**.

The alternative — having each thread iterate sequentially over its share of rows — would cause all 32 threads in a warp to request the same address on each iteration, serialising memory access completely.

```

1  /* GOOD: coalesced. Thread tid accesses row start+tid, start+tid+T, ...
2     On any given iteration, the warp accesses consecutive addresses. */
3  for (int i = start + tid; i < stop; i += T)
4     acc += A[i];
5
6  /* BAD: non-coalesced. All threads in warp access same address each iter.
7     32x worse memory bandwidth utilization. */
8  int my_start = start + tid * (stop - start) / T;
9  int my_stop  = start + (tid+1) * (stop - start) / T;
10 for (int i = my_start; i < my_stop; i++)
11     acc += A[i];

```

Listing 18. Coalesced vs non-coalesced access patterns**7. Accumulating Multiple Outputs Simultaneously****7.1. The problem: coupled outputs sharing expensive computation**

The covariance kernel does not reduce to a single scalar. It reduces to a symmetric matrix with $N_{\text{sym}} = N_{\text{eig}}(N_{\text{eig}} + 1)/2$ unique entries. Each entry shares the same weight w_i^{-1} and the same loaded vectors $\mathbf{d}_i$.

The naive solution would be to launch N_{sym} separate kernels, each computing one entry. But this loads `D` and `w` from global memory N_{sym} times separately — for $N_{\text{eig}} = 6$ that is 21 separate passes over the data instead of 1.

7.2. The multi-accumulator pattern

The solution is to carry all N_{sym} accumulators simultaneously in registers throughout the inner loop, computing all entries in a single pass:

```

1  /* N_SYM accumulators live in registers for the entire inner loop.
2     With #pragma unroll and compile-time N_SYM, the compiler allocates
3     each acc[t] to a dedicated register. */
4  float acc[N_SYM];
5  #pragma unroll
6  for (int t = 0; t < N_SYM; ++t) acc[t] = 0.0f;
7
8  for (int i = start + tid; i < stop; i += T) {
9      float wi = 1.0f / w[i];
10
11     float d[N_EIG];
12     #pragma unroll
13     for (int e = 0; e < N_EIG; ++e)
14         d[e] = D[i * N_EIG + e];    /* load row i of D once */
15
16     /* Accumulate all N_SYM upper-triangle entries in one pass.
17        wi, d[0..N_EIG-1] are loaded ONCE per row and reused N_SYM times.
18        */
19     int idx = 0;
20     #pragma unroll
21     for (int col = 0; col < N_EIG; ++col) {
22         float wd_col = wi * d[col];    /* compute once per column */
23         #pragma unroll
24         for (int row = 0; row <= col; ++row) {
25             acc[idx] += d[row] * wd_col;
26             idx++;
27         }
28     }
29 }
```

Listing 19. Multi-accumulator pattern: all outputs in one pass

The data $w[i]$ and $D[i \cdot N_{\text{EIG}} + 0 \dots N_{\text{EIG}} - 1]$ are loaded from global VRAM exactly once per row i . All N_{sym} accumulations then operate entirely on registers. This is the key reason the multi-pass approach would be wasteful: global memory bandwidth is the bottleneck, not arithmetic.

7.3. Exploiting symmetry to reduce the problem size

The outer product $\mathbf{d}_i \mathbf{d}_i^\top$ is symmetric. The entry at (j, k) equals the entry at (k, j) . Therefore we only need to compute the upper triangle — $N_{\text{sym}} = N_{\text{eig}}(N_{\text{eig}} + 1)/2$ entries instead of N_{eig}^2 :

N_{eig}	Full matrix	Upper triangle	Savings
3	9	6	33%
6	36	21	42%
10	100	55	45%
15	225	120	47%

At write-time, the full symmetric matrix is reconstructed by mirroring the off-diagonal entries:

```

1  /* The packed upper triangle in column-major order:
2     for n_eig=3: [00, 01,11, 02,12,22]
3     for n_eig=4: [00, 01,11, 02,12,22, 03,13,23,33] */
4
5  /* At write time, reconstruct the full n_eig x n_eig matrix: */
6  int idx = 0;
7  #pragma unroll
8  for (int col = 0; col < N_EIG; ++col) {
9      #pragma unroll
10     for (int row = 0; row <= col; ++row) {
11         out[row * N_EIG + col] = v[idx];    /* upper triangle */
12         out[col * N_EIG + row] = v[idx];    /* mirror to lower */
13         idx++;
14     }
15 }
16 /* For the diagonal (row==col), both writes go to the same address ---
    harmless. */

```

Listing 20. Packing upper triangle then reconstructing full matrix at write time

8. Runtime Kernel Templating with Python f-Strings

8.1. The problem: compile-time constants for loop unrolling

`#pragma unroll` only works when the loop bound is a compile-time constant. If `N_EIG` is a runtime variable (a function argument), the compiler cannot unroll the inner loops and the register-promotion benefit is lost.

The solution is to make `N_EIG` a compile-time constant by injecting it as a `#define` into the kernel source before compilation. Since NVRTC compiles the kernel string at runtime, we can generate a different kernel string for each value of `n_eig`.

8.2. The `make_kernel_source` pattern

```

1  def make_kernel_source(n_eig):
2      """Generate CUDA source for a specific n_eig at Python level."""
3      n_sym = n_eig * (n_eig + 1) // 2
4
5      # rf-string: raw (r) so backslashes are literal,
6      #           f-string (f) so {n_eig} and {n_sym} are substituted.
7      # All CUDA curly braces must be doubled: { -> {{ and } -> }}
8      source = rf'''
9      #define N_EIG {n_eig}
10     #define N_SYM {n_sym}
11
12     __device__ __forceinline__ float warp_sum(float v) {{
13         unsigned mask = 0xffffffffu;
14         v += __shfl_down_sync(mask, v, 16);
15         v += __shfl_down_sync(mask, v, 8);
16         v += __shfl_down_sync(mask, v, 4);
17         v += __shfl_down_sync(mask, v, 2);
18         v += __shfl_down_sync(mask, v, 1);
19         return v;
20     }}
21
22     extern "C" __global__
23     void my_kernel(const float* D, const float* w,

```

```

24         const int* edges, float* C_out, int nb)
25     {{
26         /* ... kernel body with #pragma unroll over N_EIG, N_SYM ... */
27         float acc[N_SYM];
28         #pragma unroll
29         for (int t = 0; t < N_SYM; ++t) acc[t] = 0.0f;
30         /* N_SYM is now a literal integer in the compiled code, so
31            the compiler fully unrolls this loop. */
32     }}
33     '''
34     return source
35
36
37 def make_kernel(n_eig, threads=128):
38     """Compile and return a kernel for a specific n_eig."""
39     source = make_kernel_source(n_eig)
40     return cp.RawKernel(source, "my_kernel")
41
42
43 # Cache compiled kernels so each n_eig is only compiled once per session
44 _kernel_cache = {}
45
46 def get_kernel(n_eig, threads=128):
47     if n_eig not in _kernel_cache:
48         _kernel_cache[n_eig] = make_kernel(n_eig, threads)
49     return _kernel_cache[n_eig]

```

Listing 21. Runtime kernel templating: injecting compile-time constants

The `rf'''...'''` prefix combines two Python string modifiers. `r` (raw string) means backslashes are treated literally, which matters if your CUDA code contains escape sequences. `f` (f-string) enables `{variable}` interpolation. Since `f` already uses `{...}` as its syntax, every literal curly brace in the CUDA C code must be doubled: `{{` produces `{` and `}}` produces `}`.

9. Putting It All Together: Building the Full Kernel

You now have all the building blocks. This section shows how they combine into the complete covariance reduction kernel, walking through each design decision in the order you would make them.

9.1. Step 1: Define the computation

Write down precisely what the kernel must compute. In our case:

$$C_b = \sum_{i \in \mathcal{S}_b} \frac{1}{w_i} \mathbf{d}_i \mathbf{d}_i^\top \quad \text{for each baseline group } b = 0, \dots, N_b - 1$$

where $\mathbf{d}_i \in \mathbb{R}^{N_{\text{eig}}}$ and C_b is $N_{\text{eig}} \times N_{\text{eig}}$ symmetric.

9.2. Step 2: Identify the parallelism

Ask: what is embarrassingly parallel? The N_b baseline groups are fully independent — assign one CUDA block per group.

Within each group, the sum over $|\mathcal{S}_b|$ data points is a reduction — assign all threads in the block to accumulate over this sum collaboratively using the strided loop pattern.

9.3. Step 3: Design the data layout

Input D: shape $(N_{\text{total}}, N_{\text{eig}})$, row-major. Row i contains $\mathbf{d}_i$. Accessed as $D[i * N_{\text{EIG}} + e]$ for component e .

Input w: shape $(N_{\text{total}},)$. Accessed as $w[i]$.

Input edges: shape $(N_b + 1,)$. CSR pointers.

Output C_out: shape $(N_b, N_{\text{eig}}, N_{\text{eig}})$, stored flat. Block b 's matrix starts at offset $b \times N_{\text{eig}}^2$.

9.4. Step 4: Plan the reduction pipeline

Following the four-phase pattern:

1. **Accumulate:** each thread accumulates N_{sym} partial sums over its assigned rows using the strided loop.
2. **Warp reduce:** call `warp_sum` on each of the N_{sym} accumulators — N_{sym} independent 32-lane reductions.
3. **Share:** lane 0 of each warp writes N_{sym} values to shared memory. Synchronise.
4. **Block reduce:** warp 0 loads the per-warp partial sums, calls `warp_sum` again, and lane 0 of warp 0 writes the final $N_{\text{eig}} \times N_{\text{eig}}$ output matrix.

9.5. Step 5: Size the shared memory

Shared memory holds $N_{\text{sym}} \times \text{num_warps}$ floats, where $\text{num_warps} = \text{threads}/32$. Compute this in Python and pass as `shared_mem`:

```
num_warps = threads // 32
shared_bytes = n_sym * num_warps * 4    # 4 bytes per float32
```

Listing 22. Shared memory sizing

9.6. Step 6: Write and annotate the kernel

```
1 extern "C" __global__
2 void cov_reduce_sym_warp(const float* __restrict__ D,
3                          const float* __restrict__ w,
4                          const int* __restrict__ edges,
5                          float* __restrict__ C_out,
6                          int nb)
7 {
8     int b = (int)blockIdx.x;
9     if (b >= nb) return;          /* bounds guard */
10
11     int start = edges[b];
12     int stop  = edges[b + 1];
```

```

13
14  /* ---- Phase 1: accumulate partial sums in registers ---- */
15  float acc[N_SYM];
16  #pragma unroll
17  for (int t = 0; t < N_SYM; ++t) acc[t] = 0.0f;
18
19  int tid = (int)threadIdx.x;
20  int T   = (int)blockDim.x;
21
22  for (int i = start + tid; i < stop; i += T) {
23      float wi = 1.0f / w[i];
24
25      float d[N_EIG];
26      #pragma unroll
27      for (int e = 0; e < N_EIG; ++e)
28          d[e] = D[i * N_EIG + e];
29
30      int idx = 0;
31      #pragma unroll
32      for (int col = 0; col < N_EIG; ++col) {
33          float wd_col = wi * d[col];
34          #pragma unroll
35          for (int row = 0; row <= col; ++row) {
36              acc[idx] += d[row] * wd_col;
37              idx++;
38          }
39      }
40  }
41
42  /* ---- Phase 2: intra-warp shuffle reduction ---- */
43  #pragma unroll
44  for (int t = 0; t < N_SYM; ++t) acc[t] = warp_sum(acc[t]);
45
46  /* ---- Phase 3: inter-warp reduction via shared memory ---- */
47  int lane      = (int)(threadIdx.x & 31);
48  int warp      = (int)(threadIdx.x >> 5);
49  int num_warps = (int)(blockDim.x >> 5);
50
51  extern __shared__ float sh[];
52
53  if (lane == 0) {
54      #pragma unroll
55      for (int t = 0; t < N_SYM; ++t)
56          sh[t * num_warps + warp] = acc[t];
57  }
58  __syncthreads();
59
60  /* ---- Phase 4: final reduction and output ---- */
61  if (warp == 0) {
62      float v[N_SYM];
63      if (lane < num_warps) {
64          #pragma unroll
65          for (int t = 0; t < N_SYM; ++t)
66              v[t] = sh[t * num_warps + lane];
67      } else {
68          #pragma unroll
69          for (int t = 0; t < N_SYM; ++t)
70              v[t] = 0.0f;          /* neutral element for unused lanes */
71      }
72  }

```

```

73     #pragma unroll
74     for (int t = 0; t < N_SYM; ++t) v[t] = warp_sum(v[t]);
75
76     if (lane == 0) {
77         /* Navigate to block b's output matrix */
78         float* out = C_out + (size_t)b * N_EIG * N_EIG;
79
80         /* Unpack upper triangle into full symmetric matrix */
81         int idx = 0;
82         #pragma unroll
83         for (int col = 0; col < N_EIG; ++col) {
84             #pragma unroll
85             for (int row = 0; row <= col; ++row) {
86                 out[row * N_EIG + col] = v[idx]; /* upper */
87                 out[col * N_EIG + row] = v[idx]; /* mirror */
88                 idx++;
89             }
90         }
91     }
92 }
93 }

```

Listing 23. The complete generalised covariance reduction kernel

9.7. Step 7: Write the Python wrapper

```

1  import cupy as cp
2  import numpy as np
3
4  _kernel_cache = {}
5
6  def make_kernel_source(n_eig):
7      n_sym = n_eig * (n_eig + 1) // 2
8      source = rf'''
9      #define N_EIG {n_eig}
10     #define N_SYM {n_sym}
11
12     __device__ __forceinline__ float warp_sum(float v) {{
13         unsigned mask = 0xffffffffu;
14         v += __shfl_down_sync(mask, v, 16);
15         v += __shfl_down_sync(mask, v, 8);
16         v += __shfl_down_sync(mask, v, 4);
17         v += __shfl_down_sync(mask, v, 2);
18         v += __shfl_down_sync(mask, v, 1);
19         return v;
20     }}
21
22     /* ... full kernel body as above ... */
23     '''
24     return source
25
26  def cov_reduce(D, w, edges, n_eig, threads=128):
27      """
28      Compute weighted outer-product sums for each baseline group.
29
30      Parameters
31      -----
32      D      : (N, n_eig) float32 CuPy array, C-contiguous
33      w      : (N,) float32 CuPy array
34      edges   : (nb+1,) int32 CuPy array (CSR pointers)

```

```

35     n_eig : int          number of eigenmodes
36     threads: int        threads per block (must be multiple of 32)
37
38     Returns
39     -----
40     out : (nb, n_eig, n_eig) float32 CuPy array
41     """
42     assert D.dtype == cp.float32 and D.flags.c_contiguous
43     assert w.dtype == cp.float32
44     assert edges.dtype == cp.int32
45
46     nb = edges.size - 1
47     n_sym = n_eig * (n_eig + 1) // 2
48     num_warps = threads // 32
49     shared_bytes = n_sym * num_warps * 4
50
51     # Compile and cache kernel for this n_eig
52     if n_eig not in _kernel_cache:
53         source = make_kernel_source(n_eig)
54         _kernel_cache[n_eig] = cp.RawKernel(source, "cov_reduce_sym_warp")
55
56     kernel = _kernel_cache[n_eig]
57     out = cp.empty((nb, n_eig, n_eig), dtype=cp.float32)
58
59     kernel(
60         (nb,),
61         (threads,),
62         (D, w, edges, out, np.int32(nb)),
63         shared_mem=shared_bytes
64     )
65     return out

```

Listing 24. The complete Python wrapper

10. Production Deployment: nvcc, Shared Libraries, and ctypes

CuPy’s `RawKernel` is excellent for development: no build system, instant iteration, everything in one Python file. But for a production pipeline — such as the CHORD data analysis system — it has real limitations: cold-start compilation latency on every new process, restricted compiler flags, inability to link against external CUDA libraries, and coupling of kernel source code to Python strings.

The production approach compiles the kernel offline with `nvcc` into a shared library (`.so`), which Python then loads via `ctypes`. The CUDA C code is completely unchanged; only the build and loading machinery differs.

10.1. The file structure

A self-contained kernel module has three files:

File	Role
<code>cov_reduce.cu</code>	CUDA kernel + CPU-side launcher function
<code>cov_reduce.h</code>	Header declaring the launcher signature
<code>cov_reduce.py</code>	Python wrapper loading the <code>.so</code> via <code>ctypes</code>

10.2. The CUDA file: kernel plus CPU launcher

The `.cu` file contains both the GPU kernel (`__global__`) and a plain C function that configures and launches it. The CPU-side launcher is what `ctypes` will call — the kernel itself is never called directly from Python.

```

1  #include <cuda_runtime.h>
2  #include <stdint.h>
3  #include "cov_reduce.h"
4
5  /*          Compile-time constants injected by nvcc -D flags
6
7      These are NOT defined here. They are passed on the nvcc command line:
8      nvcc -D N_EIG=6 -D N_SYM=21 ...
9      This is the production equivalent of the Python f-string substitution.
10     */
11
12 /*          warp_sum helper
13
14     */
15 __device__ __forceinline__ float warp_sum(float v) {
16     unsigned mask = 0xffffffffu;
17     v += __shfl_down_sync(mask, v, 16);
18     v += __shfl_down_sync(mask, v, 8);
19     v += __shfl_down_sync(mask, v, 4);
20     v += __shfl_down_sync(mask, v, 2);
21     v += __shfl_down_sync(mask, v, 1);
22     return v;
23 }
24
25 /*          GPU kernel
26
27     */
28 __global__
29 void cov_reduce_kernel(const float* __restrict__ D,
30                       const float* __restrict__ w,
31                       const int* __restrict__ edges,
32                       float* __restrict__ C_out,
33                       int nb)
34 {
35     int b = (int)blockIdx.x;
36     if (b >= nb) return;
37
38     int start = edges[b];
39     int stop  = edges[b + 1];
40
41     float acc[N_SYM];
42     #pragma unroll
43     for (int t = 0; t < N_SYM; ++t) acc[t] = 0.0f;
44
45     int tid = (int)threadIdx.x;
46     int T    = (int)blockDim.x;
47
48     for (int i = start + tid; i < stop; i += T) {
49         float wi = 1.0f / w[i];
50         float d[N_EIG];
51         #pragma unroll
52         for (int e = 0; e < N_EIG; ++e)
53             d[e] = D[i * N_EIG + e];
54     }
55 }

```

```

49     int idx = 0;
50     #pragma unroll
51     for (int col = 0; col < N_EIG; ++col) {
52         float wd_col = wi * d[col];
53         #pragma unroll
54         for (int row = 0; row <= col; ++row) {
55             acc[idx] += d[row] * wd_col;
56             idx++;
57         }
58     }
59 }
60
61 #pragma unroll
62 for (int t = 0; t < N_SYM; ++t) acc[t] = warp_sum(acc[t]);
63
64 int lane      = (int)(threadIdx.x & 31);
65 int warp_id   = (int)(threadIdx.x >> 5);
66 int num_warps = (int)(blockDim.x >> 5);
67
68 extern __shared__ float sh[];
69
70 if (lane == 0) {
71     #pragma unroll
72     for (int t = 0; t < N_SYM; ++t)
73         sh[t * num_warps + warp_id] = acc[t];
74 }
75 __syncthreads();
76
77 if (warp_id == 0) {
78     float v[N_SYM];
79     if (lane < num_warps) {
80         #pragma unroll
81         for (int t = 0; t < N_SYM; ++t)
82             v[t] = sh[t * num_warps + lane];
83     } else {
84         #pragma unroll
85         for (int t = 0; t < N_SYM; ++t)
86             v[t] = 0.0f;
87     }
88     #pragma unroll
89     for (int t = 0; t < N_SYM; ++t) v[t] = warp_sum(v[t]);
90
91     if (lane == 0) {
92         float* out = C_out + (size_t)b * N_EIG * N_EIG;
93         int idx = 0;
94         #pragma unroll
95         for (int col = 0; col < N_EIG; ++col) {
96             #pragma unroll
97             for (int row = 0; row <= col; ++row) {
98                 out[row * N_EIG + col] = v[idx];
99                 out[col * N_EIG + row] = v[idx];
100                 idx++;
101             }
102         }
103     }
104 }
105 }
106
107 /*      CPU-side launcher

```

```

108  This is a plain C function (extern "C" disables C++ name mangling).
109  ctypes will call this function directly. It receives raw pointers to
110  GPU memory --- the Python side is responsible for allocating device
111  arrays and passing their data pointers.
112
113  Parameters mirror the kernel exactly, plus launch configuration.      */
114  extern "C"
115  void launch_cov_reduce(const float* D,
116                        const float* w,
117                        const int*   edges,
118                        float*       C_out,
119                        int          nb,
120                        int          threads)
121  {
122      int n_sym      = N_SYM;           /* compile-time constant */
123      int num_warps = threads / 32;
124      int shared_bytes = n_sym * num_warps * (int)sizeof(float);
125
126      /* Launch nb blocks, each of size 'threads'.
127      The <<< >>> syntax is CUDA C only -- not valid in plain C.
128      This is why the launcher must be in a .cu file, not a .c file. */
129      cov_reduce_kernel<<<nb, threads, shared_bytes>>>(
130          D, w, edges, C_out, nb
131      );
132
133      /* Optional: check for launch errors.
134      cudaGetLastError() returns the error from the most recent kernel.
135      */
136      cudaError_t err = cudaGetLastError();
137      if (err != cudaSuccess) {
138          /* In production, propagate this error properly.
139          For now, printing is sufficient to catch launch failures. */
140          fprintf(stderr, "CUDA kernel launch error: %s\n",
141                  cudaGetErrorString(err));
142      }
143  }

```

Listing 25. cov_reduce.cu: kernel and CPU-side launcher in one file

The kernel (`__global__`) is called by the CUDA runtime, not by `ctypes`. `ctypes` calls only the CPU-side launcher. `extern "C"` on the launcher prevents C++ name mangling so that `ctypes` can find the function by its plain name "launch_cov_reduce" in the shared library.

10.3. The header file

```

1  #pragma once
2  #include <stdint.h>
3
4  #ifdef __cplusplus
5  extern "C" {
6  #endif
7
8  /* CPU-side launcher. All pointer arguments point to GPU (device) memory.
9  The caller is responsible for allocating and freeing device memory. */
10 void launch_cov_reduce(const float* D,

```

```

11         const float* w,
12         const int*   edges,
13         float*       C_out,
14         int          nb,
15         int          threads);
16
17 #ifdef __cplusplus
18 }
19 #endif

```

Listing 26. `cov_reduce.h`: declaring the launcher for external callers

10.4. Compiling with `nvcc`

```

nvcc \
  -O3 \
  -arch=sm_80 \
  -D N_EIG=6 \
  -D N_SYM=21 \
  --ptxas-options=-v \
  --shared \
  -Xcompiler -fPIC \
  -o libcov_reduce_n6.so \
  cov_reduce.cu

```

Listing 27. Compiling a versioned shared library with `nvcc`

Each flag deserves explanation:

- `-O3` — maximum optimisation level. Always use this for production kernels.
- `-arch=sm_80` — target architecture. `sm_80` is NVIDIA Ampere (A100). Use `sm_70` for Volta (V100), `sm_86` for RTX 30-series, `sm_90` for Hopper (H100). Matching the actual hardware enables architecture-specific instruction selection and avoids compatibility-mode overhead.
- `-D N_EIG=6 -D N_SYM=21` — inject compile-time constants as preprocessor definitions. This is the `nvcc` equivalent of the Python f-string `#define` injection. One `.so` file is compiled per value of `N_EIG`.
- `-ptxas-options=-v` — print register usage, shared memory usage, and local memory (spill) per kernel. The critical number is `lmem`: it must be 0 for performance-critical kernels.
- `-shared` — build a shared library rather than an executable.
- `-Xcompiler -fPIC` — pass `-fPIC` (position-independent code) to the host C++ compiler. Required for shared libraries on Linux.

Name the output `libcov_reduce_n6.so` rather than `libcov_reduce.so`. Since each `N_EIG` value produces a different binary, embedding the value in the filename lets you maintain multiple compiled versions simultaneously and load the correct one at runtime.

10.5. Loading and calling from Python via ctypes

```

1  import ctypes
2  import ctypes.util
3  import os
4  import numpy as np
5  import cupy as cp
6
7  #         Load the shared library
8
9  # Assumes the .so is in the same directory as this Python file.
10 # In a real pipeline, use an environment variable or config for the path.
11 _lib_dir = os.path.dirname(os.path.abspath(__file__))
12
13 def _load_library(n_eig):
14     """Load the compiled shared library for a specific n_eig."""
15     lib_path = os.path.join(_lib_dir, f"libcov_reduce_n{n_eig}.so")
16     if not os.path.exists(lib_path):
17         raise FileNotFoundError(
18             f"Compiled library not found: {lib_path}\n"
19             f"Run: nvcc -O3 -arch=sm_80 -D N_EIG={n_eig} "
20             f"-D N_SYM={n_eig*(n_eig+1)//2} --shared -Xcompiler -fPIC "
21             f"-o {lib_path} cov_reduce.cu"
22         )
23     lib = ctypes.CDLL(lib_path)
24
25     #         Declare the argument types of the launcher function
26
27     # ctypes needs to know the C types of each argument so it can marshal
28     # Python objects correctly. This mirrors the C signature exactly.
29     #
30     # c_void_p : a raw pointer (we pass GPU memory addresses as integers)
31     # c_int    : a 32-bit signed integer
32     lib.launch_cov_reduce.argtypes = [
33         ctypes.c_void_p, # D      (float* on GPU)
34         ctypes.c_void_p, # w      (float* on GPU)
35         ctypes.c_void_p, # edges (int*   on GPU)
36         ctypes.c_void_p, # C_out (float* on GPU)
37         ctypes.c_int,    # nb
38         ctypes.c_int,    # threads
39     ]
40     lib.launch_cov_reduce.restype = None # void return
41
42     return lib
43
44 # Cache loaded libraries per n_eig
45 _lib_cache = {}
46
47 def _get_lib(n_eig):
48     if n_eig not in _lib_cache:
49         _lib_cache[n_eig] = _load_library(n_eig)
50     return _lib_cache[n_eig]
51
52 #         Main Python wrapper
53
54 def cov_reduce(D, w, edges, n_eig, threads=128):
55     """
56     Compute batched weighted outer-product sums via compiled CUDA kernel.

```

```

55
56     Parameters
57     -----
58     D      : (N, n_eig) float32 CuPy array, C-contiguous
59     w      : (N,)      float32 CuPy array
60     edges  : (nb+1,)   int32   CuPy array
61     n_eig  : int       number of eigenmodes
62     threads: int       threads per block (multiple of 32)
63
64     Returns
65     -----
66     out : (nb, n_eig, n_eig) float32 CuPy array
67     """
68     assert D.dtype == cp.float32 and D.flags.c_contiguous
69     assert w.dtype == cp.float32
70     assert edges.dtype == cp.int32
71
72     nb = int(edges.size - 1)
73     lib = _get_lib(n_eig)
74     out = cp.empty((nb, n_eig, n_eig), dtype=cp.float32)
75
76     #           Extract raw GPU memory pointers
77
78     # CuPy arrays expose their underlying device memory address via .data.
79     # ptr
80     # This is a Python integer holding the raw VRAM address.
81     # ctypes.c_void_p accepts integers directly.
82     lib.launch_cov_reduce(
83         D.data.ptr,          # GPU address of D
84         w.data.ptr,          # GPU address of w
85         edges.data.ptr,      # GPU address of edges
86         out.data.ptr,        # GPU address of output (written by kernel)
87         ctypes.c_int(nb),
88         ctypes.c_int(threads),
89     )
90
91     #           Synchronise
92
93     # Kernel launches are asynchronous: the CPU returns immediately after
94     # issuing the launch, while the GPU executes in the background.
95     # cp.cuda.Stream.null.synchronize() blocks the CPU until the GPU has
96     # finished all pending work on the default stream.
97     # Without this, reading 'out' on the CPU (e.g. converting to NumPy)
98     # before the kernel finishes produces garbage.
99     cp.cuda.Stream.null.synchronize()
100
101     return out

```

Listing 28. `cov_reduce.py`: loading the shared library and calling via `ctypes`

10.6. What `.data.ptr` is

CuPy arrays live in GPU VRAM. The `.data.ptr` attribute returns the raw integer address of the array's first element in VRAM. Passing this to `ctypes.c_void_p` hands the GPU memory address directly to the C launcher function, which passes it to the kernel as a `float*` or `int*`.

This is safe because the kernel never dereferences these pointers on the CPU. They travel

from Python through `ctypes` through the C launcher into the kernel launch configuration, where the GPU hardware uses them to access VRAM directly.

`launch_cov_reduce` returns immediately on the CPU side, before the GPU has finished executing the kernel. If you read the output array on the CPU (e.g. `out.get()` to convert to NumPy) without synchronising first, you will read uninitialised or partial results. Always call `cp.cuda.Stream.null.synchronize()` or `out.get()` (which synchronises implicitly) before consuming output data.

10.7. Comparison: CuPy RawKernel vs `nvcc` + `ctypes`

	CuPy RawKernel	<code>nvcc</code> + <code>ctypes</code>
Compilation timing	Runtime (first call per session)	Offline (once, before deployment)
Cold-start latency	Hundreds of milliseconds	Zero (binary already compiled)
Compiler flags	Limited (NVRTC subset)	Full <code>nvcc</code> flag set
Architecture targeting	Via options tuple	Full <code>-arch/-gencode</code> control
External CUDA libraries	Not possible	Full linking support
Build system needed	No	Yes (Makefile or CMake)
Best for	Development, iteration, research	Production pipelines

11. Production Templating: `nvcc -D` and C Preprocessor Templates

The Python f-string approach in Section 8 uses Python as a text preprocessor to inject constants like `N_EIG` into the kernel source at runtime before passing the string to NVRTC. In the `nvcc` + `ctypes` workflow, the kernel source is a static `.cu` file — you cannot run a Python function to modify it at compile time in the same way. Instead you use two complementary approaches: the C preprocessor (`#define` and `-D` flags) and C++ templates.

11.1. Approach 1: `-D` flags at compile time

The simplest approach: compile a separate `.so` for each value of `N_EIG` you need, injecting the constant via the `-D` command-line flag:

```
# Compile for n_eig = 3
nvcc -O3 -arch=sm_80 -D N_EIG=3 -D N_SYM=6 \
    --shared -Xcompiler -fPIC \
    -o libcov_reduce_n3.so cov_reduce.cu
```

```

# Compile for n_eig = 6
nvcc -O3 -arch=sm_80 -D N_EIG=6 -D N_SYM=21 \
    --shared -Xcompiler -fPIC \
    -o libcov_reduce_n6.so cov_reduce.cu

# Compile for n_eig = 10
nvcc -O3 -arch=sm_80 -D N_EIG=10 -D N_SYM=55 \
    --shared -Xcompiler -fPIC \
    -o libcov_reduce_n10.so cov_reduce.cu

```

Listing 29. Compile once per N_EIG value needed

Automating this in a Makefile:

```

N_EIGS = 3 6 10 15
ARCH    = sm_80
CFLAGS  = -O3 --ptxas-options=-v --shared -Xcompiler -fPIC

TARGETS = $(foreach n, $(N_EIGS), libcov_reduce_n$(n).so)

all: $(TARGETS)

libcov_reduce_n%.so: cov_reduce.cu cov_reduce.h
    nvcc $(CFLAGS) -arch=$(ARCH) \
        -D N_EIG=$(n) \
        -D N_SYM=$(shell python3 -c "n=$(n); print(n*(n+1)//2)") \
        -o $@ $<

clean:
    rm -f $(TARGETS)

```

Listing 30. Makefile rule to build all required N_EIG variants

The kernel .cu source needs no modification at all. Wherever it uses N_EIG or N_SYM in loop bounds or array sizes, those are compile-time integer literals after preprocessing. All `#pragma unroll` directives work correctly because the bounds are constants.

11.2. Approach 2: C++ templates for compile-time generics

C++ templates are a more principled mechanism for the same idea. A templated function is a compile-time generic: you write the logic once, and the compiler instantiates a separate concrete version for each type or integer value you use it with. This is the C++ equivalent of Python's `make_kernel_source(n_eig)` function.

```

1  /* A templated __device__ function: one source, one compiled version
2     per instantiation of n_eig. */
3  template <int n_eig>
4  __device__ __forceinline__
5  void accumulate_sym(const float* __restrict__ D,
6                     const float* __restrict__ w,
7                     int start, int stop,
8                     float* __restrict__ acc)
9  {
10     constexpr int n_sym = n_eig * (n_eig + 1) / 2;
11
12     #pragma unroll
13     for (int t = 0; t < n_sym; ++t) acc[t] = 0.0f;
14

```

```

15     int tid = (int)threadIdx.x;
16     int T    = (int)blockDim.x;
17
18     for (int i = start + tid; i < stop; i += T) {
19         float wi = 1.0f / w[i];
20         float d[n_eig];
21         #pragma unroll
22         for (int e = 0; e < n_eig; ++e)
23             d[e] = D[i * n_eig + e];
24
25         int idx = 0;
26         #pragma unroll
27         for (int col = 0; col < n_eig; ++col) {
28             float wd_col = wi * d[col];
29             #pragma unroll
30             for (int row = 0; row <= col; ++row) {
31                 acc[idx] += d[row] * wd_col;
32                 idx++;
33             }
34         }
35     }
36 }
37
38 /* Templated kernel: n_eig is a compile-time integer template parameter.
39    */
40 template <int n_eig>
41 __global__
42 void cov_reduce_kernel_t(const float* __restrict__ D,
43                         const float* __restrict__ w,
44                         const int* __restrict__ edges,
45                         float* __restrict__ C_out,
46                         int nb)
47 {
48     constexpr int n_sym = n_eig * (n_eig + 1) / 2;
49
50     int b = (int)blockIdx.x;
51     if (b >= nb) return;
52
53     float acc[n_sym];
54     accumulate_sym<n_eig>(D, w, edges[b], edges[b+1], acc);
55
56     /* ... rest of reduction identical to non-templated version ... */
57 }
58
59 /*
60     Explicit instantiations
61
62     Templates are not compiled until they are instantiated. We must
63     explicitly instantiate every n_eig value we want available in the
64     shared library. The compiler generates separate machine code for each.
65 */
66 template __global__ void cov_reduce_kernel_t<3>(
67     const float*, const float*, const int*, float*, int);
68 template __global__ void cov_reduce_kernel_t<6>(
69     const float*, const float*, const int*, float*, int);
70 template __global__ void cov_reduce_kernel_t<10>(
71     const float*, const float*, const int*, float*, int);
72
73 /*
74     Dispatcher: a plain C launcher that selects the right
75     instantiation
76     ctypes calls this function. It uses a switch to select the correct

```

```

71     template instantiation based on the runtime value of n_eig. */
72     extern "C"
73     void launch_cov_reduce_t(const float* D,
74                             const float* w,
75                             const int* edges,
76                             float* C_out,
77                             int nb, int n_eig, int threads)
78     {
79         int n_sym      = n_eig * (n_eig + 1) / 2;
80         int num_warps   = threads / 32;
81         int shared_bytes = n_sym * num_warps * (int)sizeof(float);
82
83         switch (n_eig) {
84             case 3:
85                 cov_reduce_kernel_t<3><<<nb, threads, shared_bytes>>>(
86                     D, w, edges, C_out, nb);
87                 break;
88             case 6:
89                 cov_reduce_kernel_t<6><<<nb, threads, shared_bytes>>>(
90                     D, w, edges, C_out, nb);
91                 break;
92             case 10:
93                 cov_reduce_kernel_t<10><<<nb, threads, shared_bytes>>>(
94                     D, w, edges, C_out, nb);
95                 break;
96             default:
97                 fprintf(stderr,
98                     "launch_cov_reduce_t: unsupported n_eig=%d\n", n_eig);
99         }
100     }

```

Listing 31. Templating the kernel on N_EIG using C++ templates

Template parameters must be compile-time constants. `n_eig` is a runtime integer when it arrives from Python. The switch statement is the standard bridge between a runtime value and a set of compile-time instantiations — it selects which pre-compiled version to run. Each `case` is a hard-coded compile-time constant matching one instantiated template.

11.3. Comparing the two templating approaches

	-D flags (Approach 1)	C++ templates (Approach 2)
One <code>.so</code> per <code>n_eig</code> ?	Yes — one library file per value	No — all instantiations in one <code>.so</code>
Source duplication	None (same source, different compile)	None (template written once)
Adding a new <code>n_eig</code>	Recompile with new -D value	Add <code>case</code> and explicit instantiation, recompile
Build complexity	Simple Makefile loop	Slightly more C++ boilerplate
Runtime overhead	Zero (correct library already loaded)	One switch statement (negligible)
Best for	Pipelines where <code>n_eig</code> is fixed at deployment	Pipelines where <code>n_eig</code> may vary at runtime

11.4. The `constexpr` keyword

Inside a templated function, template integer parameters are compile-time constants. The keyword `constexpr` explicitly marks a variable as a compile-time constant, which is required for it to appear as a loop bound that `#pragma unroll` and fixed-size array declarations can use:

```
template <int n_eig>
__device__ void my_func(...) {
    /* n_eig is already a compile-time constant (template parameter).
       n_sym is derived from it, but the compiler needs the constexpr
       keyword to treat it as a compile-time constant too. */
    constexpr int n_sym = n_eig * (n_eig + 1) / 2;

    float acc[n_sym];           /* OK: n_sym is constexpr */
    #pragma unroll
    for (int t = 0; t < n_sym; ++t) acc[t] = 0.0f; /* OK: unrollable */
}
```

Listing 32. `constexpr`: marking derived quantities as compile-time constants

Without `constexpr`, the compiler might treat `n_sym` as a runtime variable even though its value is deterministic at compile time, preventing unrolling and register promotion.

12. Performance Checklist and Common Mistakes

12.1. Checklist before calling a kernel production-ready

1. **Memory coalescing:** does the strided loop ensure that threads in a warp access consecutive addresses on each iteration?
2. **Register residency:** are your accumulators declared with compile-time-known sizes

and unrolled initialisation so the compiler keeps them in registers, not local memory?

3. **No register spilling:** compile with `-ptxas-options=-v` and check that `lmem` is 0. Non-zero `lmem` means register spilling to VRAM.
4. **Synchronisation:** every write to shared memory must be followed by `__syncthreads()` before any thread reads from it.
5. **Neutral elements:** any warp lane that does not have a meaningful partial sum must be initialised to the identity element (0 for sum, $-\infty$ for max, 1 for product) before the warp reduction.
6. **Bounds guard:** the first lines of a kernel should guard against blocks or threads with indices beyond your data.
7. **No divergence in hot loops:** avoid `if` statements inside the inner accumulation loop that some threads satisfy and others do not.

12.2. Common mistakes

The most common mistake from CPU programmers: having one thread do all the work while the rest idle. If you see `if (threadIdx.x == 0) { for loop over everything }`, something is wrong.

Reading shared memory immediately after writing it without a barrier is a race condition. The write from one warp may not be visible to another warp yet. `__syncthreads()` is not optional.

Iterating sequentially per-thread instead of with stride `T` can reduce effective memory bandwidth by 32x. Always check that consecutive threads access consecutive addresses.

Unrolling very large loops forces the compiler to keep many variables live simultaneously, potentially exceeding register capacity and spilling to slow local memory. Always check `lmem` in `-ptxas-options=-v` output after adding unroll pragmas.

13. Quick Reference: Kernel Design Recipes

Recipe 1: Element-wise kernel

Use when: each output element is computed independently from one or a few input elements.

Pattern: `int i = blockIdx.x * blockDim.x + threadIdx.x; if (i >= N) return; C[i] = f(A[i], B[i]);`

Grid: `ceil(N / threads)` blocks.

Block: 128 or 256 threads.

Recipe 2: Single reduction

Use when: reduce one array to one scalar.

Pattern: strided accumulation → `warp_sum` → shared memory → `warp_sum` on warp 0 → lane 0 writes.

Grid: 1 block (or multiple blocks with a second-pass kernel).

Shared mem: `num_warps * sizeof(float)`.

Recipe 3: Batched irregular reduction (CSR pattern)

Use when: many independent reductions over variable-length segments.

Pattern: `edges` array + one block per segment + same four-phase reduction as Recipe 2.

Grid: `nb` blocks (one per segment).

Block: 128 threads (tune for your segment size distribution).

Shared mem: `n_outputs * num_warps * sizeof(float)`.

Recipe 4: Multi-output batched reduction

Use when: each segment must produce multiple coupled outputs (e.g., all entries of a matrix) that share expensive per-row computation.

Pattern: Recipe 3, but with a register array `acc[N_OUT]` holding all outputs simultaneously. Apply `#pragma unroll` on the initialisation and reduction loops. Use compile-time `N_OUT` via `#define` injected from Python.

Key: data loaded from VRAM exactly once per row, all outputs accumulated in the same pass.